

IMS – Inventory Management System

Technical Documentation

Table of Contents

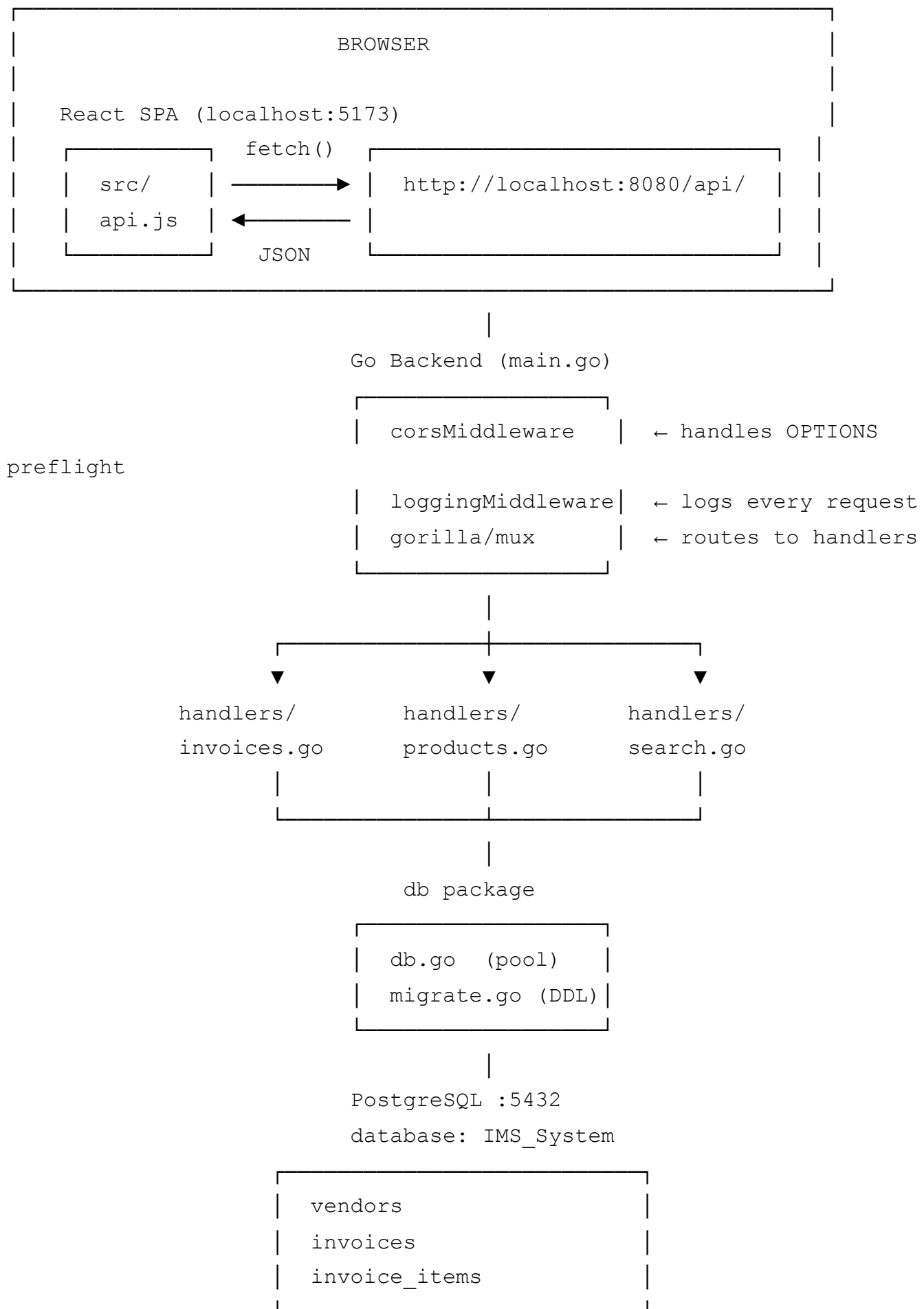
- 1. [System Overview](#)
 - 2. [Architecture](#)
 - 3. [Project File Structure](#)
 - 4. [Database Schema](#)
 - 5. [Data Structures](#)
 - 6. [API Reference](#)
 - 7. [Frontend → Backend Communication](#)
 - 8. [Backend → Database Communication](#)
 - 9. [Full Request Lifecycle Flows](#)
 - 10. [Frontend Component Map](#)
 - 11. [Vendor Deduplication Logic](#)
-

1. System Overview

IMS is a web-based inventory management system for tracking vendors, purchase invoices, and invoice line items (products). Employees submit invoices through a form; the backend automatically handles vendor deduplication, invoice creation, and product storage in a single atomic transaction.

Layer	Technology	Port
Frontend	React 19 + Tailwind v4	5173
Backend	Go (net/http + gorilla/mux)	8080
Database	PostgreSQL	5432

2. Architecture



3. Project File Structure

Backend – IMS/Backend/

Backend/

└─ main.go

Entry point. Connects DB, runs migrations, registers all routes, starts HTTP server on :8080.

Applies corsMiddleware and loggingMiddleware.

└─ go.mod

Go module definition. Dependencies:

- github.com/gorilla/mux (HTTP router)
- github.com/lib/pq (PostgreSQL

driver)

└─ db/

└─ db.go

Opens and holds the global *sql.DB connection pool.

Reads config from env vars with fallbacks:

DB_HOST=localhost, DB_PORT=5432,
DB_USER=postgres, DB_PASSWORD=1234,
DB_NAME=IMS_System

└─ migrate.go

Runs CREATE TABLE IF NOT EXISTS for all three

tables and ALTER TABLE to widen numeric columns.

Called once on every server startup

(idempotent).

└─ models/

└─ models.go

All Go structs with JSON tags.
Defines: Vendor, Invoice, InvoiceItem,
CreateInvoiceRequest, InvoiceItemInput,
UpdateProductRequest, SearchResult,

InvoiceWithItems.

└─ handlers/

└─ invoices.go

Handles: GET /api/invoices, POST

/api/invoices,

GET /api/invoices/:id, DELETE

/api/invoices/:id.

Contains findOrCreateVendor() logic.

└─ products.go	Handles: PUT /api/products/:id, DELETE /api/products/:id.
└─ search.go	Handles: GET /api/search?q= Searches invoice_items and invoices tables using LIKE pattern matching.
└─ helpers.go	Shared respondJSON() and respondError() utilities.

Frontend – IMS/Frontend/src/

src/	
└─ main.jsx	React DOM entry point. Mounts <App />.
└─ App.jsx	Root component. Sets up BrowserRouter and renders <Home /> at route "/".
└─ api.js	Single HTTP client module. All fetch() calls to the backend go through here. Base URL: http://localhost:8080/api Exports: api.listInvoices, createInvoice, getInvoice, deleteInvoice, updateProduct, deleteProduct, search.
└─ components/	
└─ home.jsx	Main page. Owns the global invoices[] state. Fetches from DB on mount via api.listInvoices().
└─ VendorGroup.jsx	Groups invoices by vendor_id using groupByVendor().
└─ InvoiceBlock.jsx	Renders: navbar, SearchBar, VendorGroup list, Add Invoice modal, toast notifications.
└─ ProductDetailModal.jsx	Renders one vendor card containing all its invoices. Contains InvoiceBlock (inner component) for each invoice. Each product row is clickable → opens ProductDetailModal. Handles invoice delete confirmation.

— InvoiceForm.jsx	2-step form inside a Modal:
	Step 1: Invoice header fields
	Step 2: One or more product rows
	On submit → calls <code>api.createInvoice()</code> .
— ProductDetailModal.jsx	Full-screen modal showing all product
fields.	
	Has 3 internal modes: view / edit /
confirm-delete.	
	Edit mode embeds <code>EditProductForm</code> .
	Calls <code>api.updateProduct()</code> and
<code>api.deleteProduct()</code> .	
— EditProductForm.jsx	Form for editing product fields.
	Used inside <code>ProductDetailModal</code> .
	Calls <code>api.updateProduct()</code> on submit.
— SearchBar.jsx	Debounced search input (400ms delay).
	Calls <code>api.search()</code> on each keystroke.
	Passes results up to <code>home.jsx</code> via
<code>onResults()</code> .	
— SearchResults.jsx	Renders search hits split into Invoices /
Products.	
	Product rows are clickable → opens
<code>ProductDetailModal</code> .	
— Modal.jsx	Generic overlay wrapper. Closes on Escape key
	or backdrop click. Supports <code>wide</code> prop for
wider panels.	
— Field.jsx	Reusable labelled <code><input></code> or <code><select></code>
component.	
	Used in <code>InvoiceForm</code> and <code>EditProductForm</code> .

4. Database Schema

Table: vendors

Column	Type	Notes
id	SERIAL PRIMARY KEY	
vendor_name	VARCHAR(255) NOT NULL	
gstin_uin	VARCHAR(50)	Used as dedup key
address	TEXT	
phone	VARCHAR(20)	
email	VARCHAR(255)	
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Indexes: `idx_vendors_name` on `vendor_name` , `idx_vendors_gstin` on `gstin_uin`

Table: `invoices`

Column	Type	Notes
id	SERIAL PRIMARY KEY	
vendor_id	INTEGER	FK → vendors(id) ON DELETE CASCADE
invoice_num	VARCHAR(100)	
invoice_date	DATE	
internal_ref	VARCHAR(100)	
total_tax	NUMERIC(12,2)	Sum of all item total_tax
total_amount	NUMERIC(12,2)	Sum of all item total_amount
invoice_img_url	TEXT	
registered_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

Indexes: `idx_invoices_vendor` on `vendor_id`

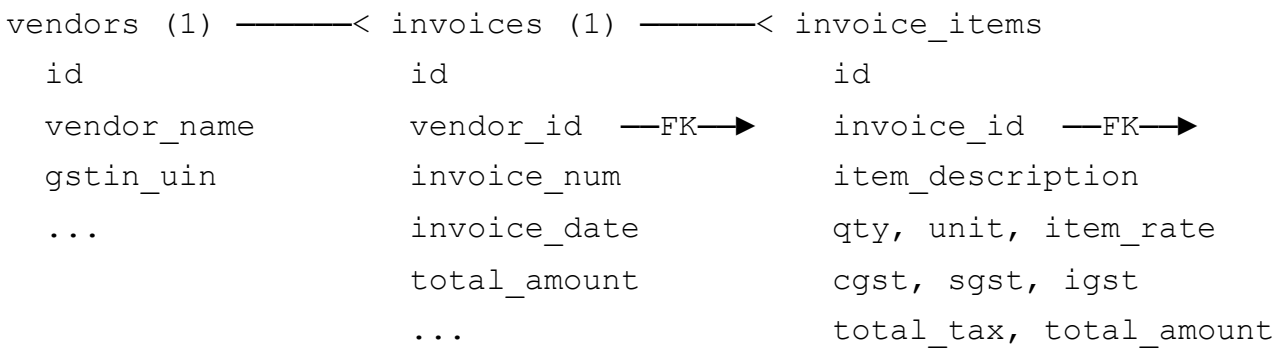
Table: `invoice_items`

Column	Type	Notes
id	SERIAL PRIMARY KEY	
invoice_id	INTEGER	FK → invoices(id) ON DELETE CASCADE

Column	Type	Notes
item_description	TEXT	
ledger_account	VARCHAR(255)	
qty	NUMERIC(12,2)	
unit	VARCHAR(50)	
item_rate	NUMERIC(15,2)	
amount	NUMERIC(15,2)	qty × item_rate
hsn_sac	VARCHAR(50)	
cgst	NUMERIC(12,2)	% rate
sgst	NUMERIC(12,2)	% rate
igst	NUMERIC(12,2)	% rate
total_tax	NUMERIC(15,2)	Computed tax amount in ₹
total_amount	NUMERIC(15,2)	amount + total_tax

Indexes: `idx_items_invoice` on `invoice_id`

Entity Relationship



5. Data Structures

All data exchanged between frontend and backend is JSON. The Go structs in `models/models.go` define the canonical shapes.

Vendor (DB record)

```
{
  "id": 4,
  "vendor_name": "ABC Traders",
  "gstn_uin": "07ABCDE1234F1Z5",
  "address": "",
  "phone": "",
  "email": "",
  "created_at": "2026-05-14T11:00:00Z"
}
```

InvoiceItem (DB record / API response)

```
{
  "id": 12,
  "invoice_id": 7,
  "item_description": "HP Laptop",
  "ledger_account": "Electronics",
  "qty": 2,
  "unit": "pcs",
  "item_rate": 55000,
  "amount": 110000,
  "hsn_sac": "8471",
  "cgst": 9,
  "sgst": 9,
  "igst": 0,
  "total_tax": 19800,
  "total_amount": 129800
}
```

InvoiceWithItems (primary API response shape)

```
{
  "id": 7,
  "vendor_id": 4,
  "vendor_name": "ABC Traders",
  "gstn_uin": "07ABCDE1234F1Z5",
  "invoice_num": "INV-102",
  "invoice_date": "2026-05-14",
  "internal_ref": "INT-5002",
  "total_tax": 20250,
```



```
"total_amount": 132750,  
"invoice_img_url": "bucket/invoices/inv102.png",  
"registered_at": "2026-05-14T11:18:00Z",  
"items": [ ...InvoiceItem[] ]  
}
```

CreateInvoiceRequest (POST /api/invoices body)

```
{  
  "invoice_date": "2026-05-14",  
  "vendor_name": "ABC Traders",  
  "gst_in_uin": "07ABCDE1234F1Z5",  
  "invoice_num": "INV-102",  
  "internal_ref": "INT-5002",  
  "invoice_img_url": "bucket/invoices/inv102.png",  
  "items": [  
    {  
      "item_description": "HP Laptop",  
      "ledger_account": "Electronics",  
      "qty": 2,  
      "unit": "pcs",  
      "item_rate": 55000,  
      "amount": 110000,  
      "hsn_sac": "8471",  
      "cgst": 9,  
      "sgst": 9,  
      "igst": 0,  
      "total_tax": 19800,  
      "total_amount": 129800  
    }  
  ]  
}
```

UpdateProductRequest (PUT /api/products/:id body)

```
{  
  "item_description": "HP Laptop Pro",  
  "qty": 3,  
  "item_rate": 53000,  
  "amount": 159000,  
  "cgst": 9,
```

```
"sgst": 9,
"igst": 0,
"total_tax": 28620,
"total_amount": 187620
}
```

Only non-zero fields are applied. Omitted fields retain their current DB value.

SearchResponse (GET /api/search response)

```
{
  "query": "laptop",
  "count": 2,
  "results": [
    {
      "type": "product",
      "payload": { ...InvoiceItem }
    },
    {
      "type": "invoice",
      "payload": { ...InvoiceWithItems (no items array) }
    }
  ]
}
```

Error Response (all endpoints on failure)

```
{
  "error": "Human-readable error message"
}
```

6. API Reference

Base URL: `http://localhost:8080/api`

All requests and responses use `Content-Type: application/json`.

GET /api/invoices

Returns all invoices with their items, ordered newest first.

Response 200 : `InvoiceWithItems[]`

```
[
  {
    "id": 7,
    "vendor_id": 4,
    "vendor_name": "ABC Traders",
    "gstn_uin": "07ABCDE1234F1Z5",
    "invoice_num": "INV-102",
    "invoice_date": "2026-05-14T00:00:00Z",
    "internal_ref": "INT-5002",
    "total_tax": 20250,
    "total_amount": 132750,
    "invoice_img_url": "",
    "registered_at": "2026-05-14T11:18:00Z",
    "items": [ ...InvoiceItem[] ]
  }
]
```

POST `/api/invoices`

Creates a vendor (or reuses existing by GSTIN), creates the invoice, and inserts all items in a single transaction.

Request body: `CreateInvoiceRequest`

Response 201 : `InvoiceWithItems` (the newly created invoice with all items)

Response 400 : Missing `vendor_name` or empty `items` array

Response 500 : DB error

GET `/api/invoices/:id`

Returns a single invoice with all its items.

Response 200 : `InvoiceWithItems`

Response 404 : Invoice not found

DELETE /api/invoices/:id

Deletes the invoice. All invoice_items are removed automatically via `ON DELETE CASCADE`.

Response 200 :

```
{ "message": "Invoice and all its items deleted successfully" }
```

Response 404 : Invoice not found

PUT /api/products/:id

Updates editable fields on a single invoice item. Only non-zero fields in the request body are applied.

Request body: `UpdateProductRequest`

Response 200 : Updated `InvoiceItem`

Response 404 : Product not found

DELETE /api/products/:id

Removes a single invoice item without affecting the parent invoice.

Response 200 :

```
{ "message": "Product deleted successfully" }
```

Response 404 : Product not found

GET /api/search?q=<query>

Case-insensitive LIKE search across products and invoices.

Searched fields:

Table	Fields searched
invoice_items	item_description, ledger_account, hsn_sac, qty, amount

Table	Fields searched
invoices	invoice_num, internal_ref, invoice_date
vendors	vendor_name, gstin_uin

Response 200 : `SearchResponse`

Response 400 : Missing `q` parameter

7. Frontend → Backend Communication

All HTTP calls are centralised in `src/api.js`. Components never call `fetch()` directly.

Component	api.js call	HTTP Request
home.jsx (on mount)	<code>api.listInvoices()</code>	GET <code>/api/invoices</code>
home.jsx (refresh btn)	<code>api.listInvoices()</code>	GET <code>/api/invoices</code>
InvoiceForm.jsx	<code>api.createInvoice(body)</code>	POST <code>/api/invoices</code>
VendorGroup.jsx	<code>api.deleteInvoice(id)</code>	DELETE <code>/api/invoices/:id</code>
ProductDetailModal.jsx	<code>api.updateProduct(id,b)</code>	PUT <code>/api/products/:id</code>
ProductDetailModal.jsx	<code>api.deleteProduct(id)</code>	DELETE <code>/api/products/:id</code>
SearchBar.jsx	<code>api.search(q)</code>	GET <code>/api/search?q=...</code>
SearchResults.jsx	(opens <code>ProductDetailModal</code> , which calls above)	

Request Headers (all requests)

Content-Type: `application/json`

CORS

The backend sets these headers on every response:

Access-Control-Allow-Origin: `*`

Access-Control-Allow-Methods: `GET, POST, PUT, DELETE, OPTIONS`

Access-Control-Allow-Headers: `Content-Type`

OPTIONS preflight requests are answered with `204 No Content` before reaching the router.

8. Backend → Database Communication

The backend uses `database/sql` with the `lib/pq` PostgreSQL driver. A single global connection pool (`db.DB`) is shared across all handlers.

Connection config (`db/db.go`)

Env Var	Default
DB_HOST	localhost
DB_PORT	5432
DB_USER	postgres
DB_PASSWORD	1234
DB_NAME	IMS_System

Query map

Handler function	SQL operations
ListInvoices	SELECT invoices JOIN vendors; SELECT invoice_items per invoice
CreateInvoice	BEGIN; SELECT vendors (GSTIN lookup); INSERT vendors (if new); INSERT invoices; INSERT invoice_items ×N; COMMIT
GetInvoice	SELECT invoices JOIN vendors WHERE id; SELECT invoice_items
DeleteInvoice	DELETE invoices WHERE id (CASCADE removes items)
UpdateProduct	SELECT invoice_items WHERE id; UPDATE invoice_items WHERE id
DeleteProduct	DELETE invoice_items WHERE id
Search	SELECT invoice_items WHERE LIKE; SELECT invoices JOIN vendors WHERE LIKE

9. Full Request Lifecycle Flows

Flow A – Load page (GET /api/invoices)

Browser loads localhost:5173

|



home.jsx useEffect fires

|



api.listInvoices()

| GET /api/invoices



loggingMiddleware → corsMiddleware → mux router

|



handlers.ListInvoices()

|

└─▶ SELECT invoices JOIN vendors ORDER BY registered_at DESC

| → []InvoiceWithItems (no items yet)

|

└─▶ for each invoice:

 SELECT invoice_items WHERE invoice_id = \$1

 → appends items to each invoice

|



respondJSON(200, []InvoiceWithItems)

|



home.jsx setInvoices(data)

|



useMemo groupByVendor(invoices)

→ groups by vendor_id into VendorGroup[]

|



Renders VendorGroup cards

Flow B – Create Invoice (POST /api/invoices)

User fills InvoiceForm (Step 1: header, Step 2: products)

|



api.createInvoice(payload)

| POST /api/invoices

```

| Body: CreateInvoiceRequest
▼
handlers.CreateInvoice()
|
|→ BEGIN transaction
|
|→ findOrCreateVendor(vendorName, gstinUIN)
|   |
|   |→ if gstinUIN != "":
|   |   SELECT id FROM vendors WHERE gstin_uin = $1
|   |   |→ found → return existing vendor_id
|   |   |→ not found → INSERT vendors → return new vendor_id
|   |→ if gstinUIN == "":
|   |   INSERT vendors → return new vendor_id
|
|→ INSERT invoices (vendor_id, invoice_num, ...) RETURNING id
|
|→ for each item in request:
|   INSERT invoice_items (invoice_id, ...) RETURNING id
|
|→ COMMIT
|
▼
respondJSON(201, InvoiceWithItems)
|
▼
home.jsx handleInvoiceCreated(invoice)
  → setInvoices([newInvoice, ...prev])
  → groupByVendor re-runs via useMemo
  → UI updates instantly (no re-fetch needed)

```

Flow C – Edit Product (PUT /api/products/:id)

```

User clicks product row in VendorGroup
|
▼
ProductDetailModal opens (mode = "view")
|
User clicks Edit button
|

```


▼

mode = "edit" → EditProductForm renders

|

User changes fields → clicks Save

|

▼

api.updateProduct(id, payload)

| PUT /api/products/:id

| Body: UpdateProductRequest (only changed fields)

▼

handlers.UpdateProduct()

|

└─▶ SELECT invoice_items WHERE id = \$1 (fetch current values)

└─▶ merge: non-zero request fields override current values

└─▶ UPDATE invoice_items SET ... WHERE id = \$1

|

▼

respondJSON(200, InvoiceItem)

|

▼

ProductDetailModal handleUpdated(updated)

→ mode = "view", shows updated data

→ onUpdated(updated) bubbles to VendorGroup → home.jsx

→ home.jsx updates invoices[] state in-place

Flow D – Delete Invoice (DELETE /api/invoices/:id)

User clicks Delete on invoice header in VendorGroup

|

▼

Confirm modal appears

|

User confirms

|

▼

api.deleteInvoice(id)

| DELETE /api/invoices/:id

▼

handlers.DeleteInvoice()

|

└─▶ DELETE FROM invoices WHERE id = \$1

PostgreSQL CASCADE → also deletes all invoice_items

```
|  
▼  
respondJSON(200, { message: "..."} )  
|  
▼  
VendorGroup onInvoiceDeleted(id)  
  → home.jsx setInvoices(prev.filter(inv => inv.id !== id))  
  → if vendor has no more invoices, VendorGroup disappears from UI
```

Flow E – Search (GET /api/search?q=)

User types in SearchBar (debounced 400ms)

```
|  
▼  
api.search(q)  
  | GET /api/search?q=laptop  
  ▼  
handlers.Search()  
  |  
  |→ SELECT invoice_items WHERE  
  |   LOWER(item_description) LIKE '%laptop%'  
  |   OR LOWER(ledger_account) LIKE '%laptop%'  
  |   OR LOWER(hsn_sac) LIKE '%laptop%'  
  |   OR CAST(qty AS TEXT) LIKE '%laptop%'  
  |   OR CAST(amount AS TEXT) LIKE '%laptop%'  
  |   LIMIT 50  
  |  
  |→ SELECT invoices JOIN vendors WHERE  
  |   LOWER(invoice_num) LIKE '%laptop%'  
  |   OR LOWER(internal_ref) LIKE '%laptop%'  
  |   OR LOWER(vendor_name) LIKE '%laptop%'  
  |   OR LOWER(gstin_uin) LIKE '%laptop%'  
  |   OR CAST(invoice_date AS TEXT) LIKE '%laptop%'  
  |   LIMIT 50  
  |  
  ▼  
respondJSON(200, { query, count, results: SearchResult[] })  
  |  
  ▼  
SearchBar onResults(data) → home.jsx setSearchData / setIsSearching
```

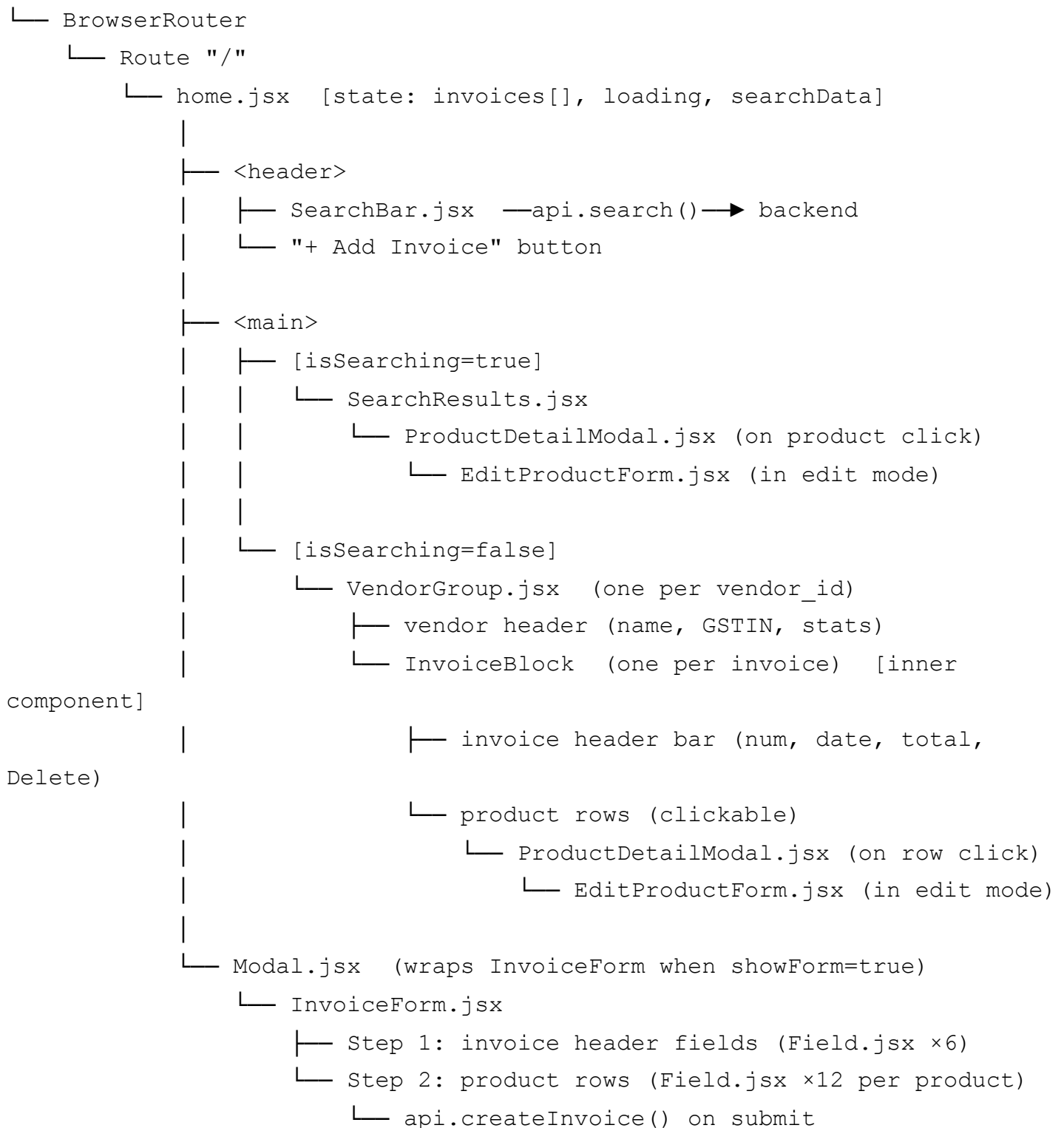


SearchResults renders:

- Invoice hits as info cards
- Product hits as clickable rows → ProductDetailModal on click

10. Frontend Component Map

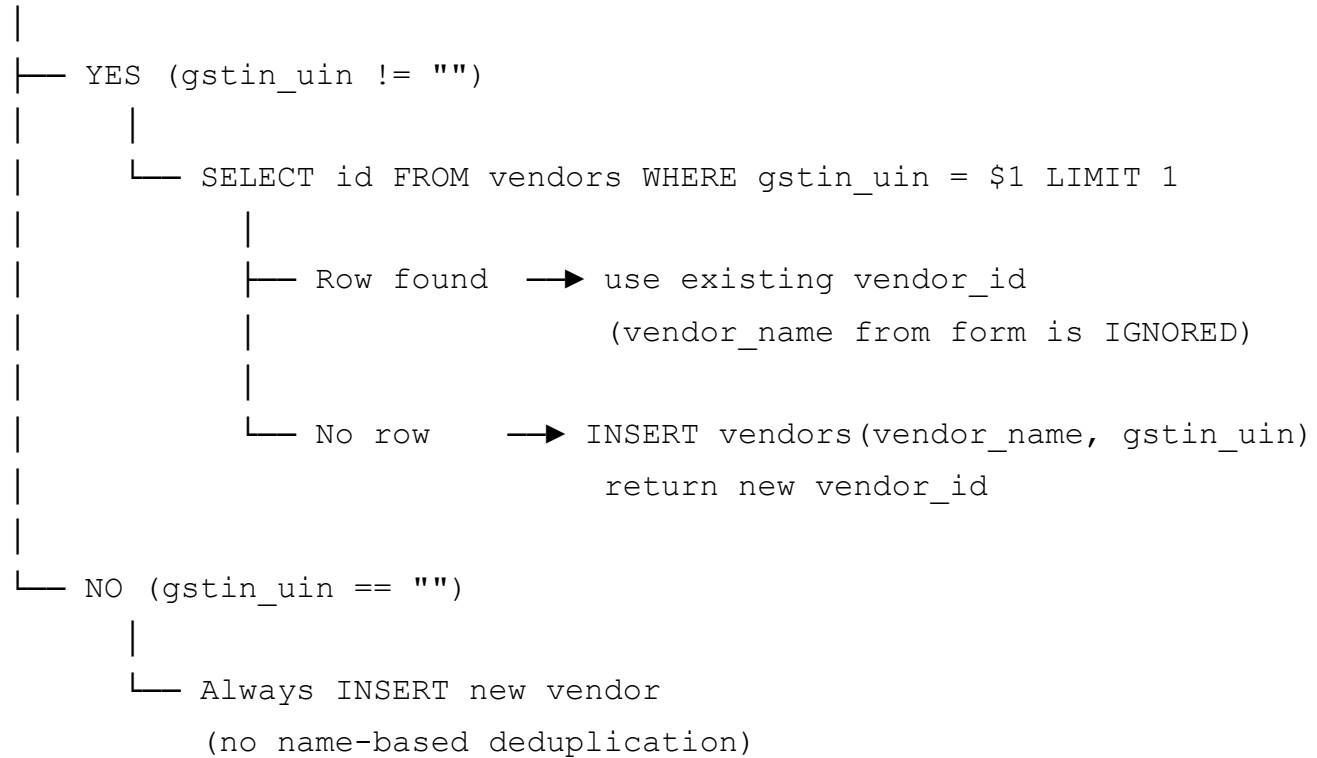
App.jsx



11. Vendor Deduplication Logic

Located in `handlers/invoices.go` → `findOrCreateVendor()` .

Incoming request has `gstn_uin`?



Key behaviour: The canonical vendor name shown in the UI is always the name that was stored when the vendor was first created. Subsequent invoices with the same GSTIN but a different name spelling will be linked to the original vendor record – the new name is not saved.